SC1004 Linear Algebra for Computing
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1 Vectors and Geometry **1**

1.1 What Is a Vector? . . . 1

1.2 Length, Dot Product, and Angle . . . 2

1.3 Lines and Planes . . . 3

1.4 Cross Product and Volume . . . 4

1.5 Inequalities and Geometry . . . 4

1.6 Chapter Notes . . . 4

1.7 Exercises . . . 5

2 Matrices **6**

2.1 Anatomy of a Matrix . . . 6

2.2 Matrix Multiplication . . . 7

2.2.1 Four Ways to See $A\mathbf{x}$ . . . 7

2.3 Transpose and Special Matrices . . . 8

2.4 Matrix Inverses . . . 8

2.5 Linear Maps and Matrix Representation . . . 9

2.6 Chapter Notes . . . 9

2.7 Exercises . . . 9

3 Linear Systems and Gaussian Elimination **11**

3.1 Systems as Matrices . . . 11

3.2 Row Operations and Echelon Form . . . 12

3.3 Solution Sets: Pivots vs. Free Variables . . . 12

3.4 LU Factorisation . . . 13

3.5 Rank and Its Geometry . . . 13
3.6 Computational Cost and Pivoting . . . 14
3.7 Chapter Notes . . . 14
3.8 Exercises . . . 14

4 Vector Spaces **16**

4.1 Why Abstract? . . . 16
4.2 The Axioms . . . 17
4.3 Subspaces . . . 17
4.4 Four Fundamental Subspaces of a Matrix . . . 17
4.5 Span and Spanning Sets . . . 18
4.6 Sums and Direct Sums . . . 18
4.7 Coordinates Revisited . . . 19
4.8 Linear Combinations and Spanning Revisited . . . 19
4.9 Chapter Notes . . . 19
4.10 Exercises . . . 19

5 Linear Independence, Basis, and Dimension **21**

5.1 Linear Independence . . . 21
5.2 Span, Basis, Dimension . . . 22
5.3 Bases for the Four Subspaces; Rank–Nullity . . . 22
5.4 Exchange and Dimension Uniqueness . . . 23
5.5 Matrix Rank = Row Rank = Column Rank . . . 24
5.6 Change of Basis and Similarity Preview . . . 24
5.7 Chapter Notes . . . 24
5.8 Exercises . . . 24

6 Orthogonality and Least Squares **25**

6.1 Orthogonality in $\mathbb{R}^n$ . . . 25
6.2 Projections onto Subspaces . . . 26
6.3 Gram–Schmidt and QR . . . 26

6.4 Least Squares . . . 27
6.5 Orthogonal Complements and the Four Subspaces . . . 28
6.6 Best Approximation and the Projection Theorem . . . 28
6.7 Numerical Stability: QR vs. Normal Equations . . . 28
6.8 Chapter Notes . . . 29
6.9 Exercises . . . 29

7 Determinants **30**

7.1 Area, Volume, and the 2×2 Determinant . . . 30
7.2 Axioms and Row-Operation Rules . . . 31
7.3 Cofactor Expansion and Adjugate . . . 31
7.4 Determinant and Eigenvalues . . . 32
7.5 Multiplicativity and Invertibility Revisited . . . 32
7.6 Determinant, Rank, and Eigenvalues . . . 32
7.7 Cramer's Rule and Its Cost . . . 33
7.8 Chapter Notes . . . 33
7.9 Exercises . . . 33

8 Eigenvalues, Eigenvectors, and Diagonalization **35**

8.1 Eigenpairs: Directions That Stay Put . . . 35
8.2 Eigenspaces and Multiplicities . . . 36
8.3 Diagonalization . . . 37
8.4 Symmetric Matrices and the Spectral Theorem . . . 38
8.5 Complex Eigenvalues and Real Blocks . . . 38
8.6 Markov Chains and Steady States . . . 38
8.7 Singular Value Decomposition (Preview) . . . 38
8.8 Chapter Notes . . . 39
8.9 Exercises . . . 39

Chapter 1

Vectors and Geometry

"A vector is not just a list of numbers — it is an arrow that knows where to push." This chapter gives that arrow a home: from points on a map to n-dimensional space, with pictures before formulas.

From zero: no prior linear algebra assumed. We start with a dot on paper, then an arrow, then coordinates. Every notion — length, angle, line, plane — is drawn before it is defined. By the end you will do geometry with algebra.

Learning Outcomes

After this chapter you will be able to:

(L1) represent vectors geometrically and algebraically in $\mathbb{R}^2$, $\mathbb{R}^3$, and $\mathbb{R}^n$;

(L2) compute dot products, norms, and angles and interpret them geometrically;

(L3) write parametric equations of lines and planes;

(L4) use vector geometry to solve distance and projection problems;

(L5) translate between pictures, coordinates, and code (NumPy).

1.1 What Is a Vector?

Intuition: stand on a map. A *point* says where you are. A *vector* says how to move: "3 east, 2 north." That move has a length and a direction, but no fixed location — sliding the arrow keeps the same vector.

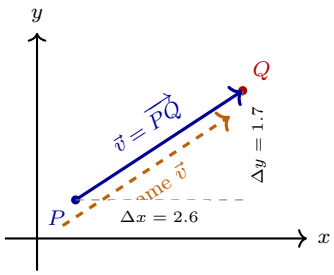

Figure 1.1: A vector as a free arrow. $\overrightarrow{PQ}$ and its translate have the same components $(\Delta x, \Delta y)$.

Definition 1.1 (Vector in $\mathbb{R}^n$)**.** A *vector* $\mathbf{v} \in \mathbb{R}^n$ is an ordered n-tuple

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}, \quad v_i \in \mathbb{R}.$$

We write $\mathbf{0}$ for the zero vector and $-\mathbf{v}$ for the opposite arrow. Two arrows represent the same vector iff their component differences agree.

Addition is tip-to-tail; scalar multiplication stretches.

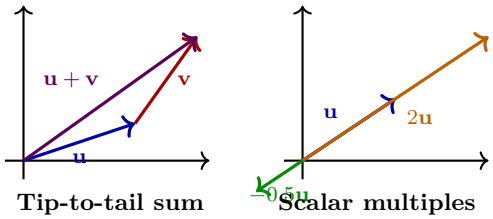

Figure 1.2: Left: $\mathbf{u} + \mathbf{v}$ (parallelogram rule). Right: $c\mathbf{u}$ scales length by $|c|$, flips if $c < 0$.

Algebraically,

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} u_1 + v_1 \\ \vdots \\ u_n + v_n \end{bmatrix}, \quad c\mathbf{u} = \begin{bmatrix} cu_1 \\ \vdots \\ cu_n \end{bmatrix}.$$

Properties: commutativity, associativity, distributivity — all inherited from $\mathbb{R}$.

1.2 Length, Dot Product, and Angle

How long is an arrow? Pythagoras in n dimensions.

Definition 1.2 (Norm and dot product)**.** For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$,

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + \cdots + u_n v_n, \qquad \|\mathbf{u}\| = \sqrt{\mathbf{u} \cdot \mathbf{u}} = \sqrt{u_1^2 + \cdots + u_n^2}.$$

Distance between tips: $\|\mathbf{u} - \mathbf{v}\|$.

Theorem 1.1 (Cauchy–Schwarz and angle)**.** For any $\mathbf{u}, \mathbf{v}$,

$$|\mathbf{u} \cdot \mathbf{v}| \le \|\mathbf{u}\|\,\|\mathbf{v}\|,$$

with equality iff one is a scalar multiple of the other. If both nonzero, there is a unique $\theta \in [0, \pi]$ with

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\|\,\|\mathbf{v}\| \cos\theta,$$

and θ is the geometric angle between them.

Sketch. Consider $\|\mathbf{u} - t\mathbf{v}\|^2 \ge 0$ for all $t \in \mathbb{R}$; its discriminant must be ≤ 0, giving Cauchy–Schwarz. Define θ via $\cos\theta = (\mathbf{u} \cdot \mathbf{v})/(\|\mathbf{u}\|\|\mathbf{v}\|)$. □

Corollary: $\mathbf{u} \perp \mathbf{v}$ (orthogonal) iff $\mathbf{u} \cdot \mathbf{v} = 0$.

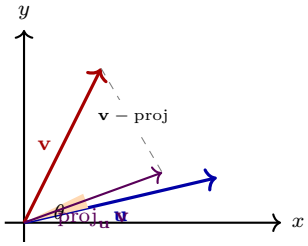

Figure 1.3: Angle θ via the dot product; projection of $\mathbf{v}$ onto $\mathbf{u}$ is the shadow along $\mathbf{u}$.

Projection: the shadow of $\mathbf{v}$ on $\mathbf{u} \neq \mathbf{0}$ is

$$\operatorname{proj}_{\mathbf{u}} \mathbf{v} = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}}\,\mathbf{u}, \qquad \|\operatorname{proj}_{\mathbf{u}} \mathbf{v}\| = \|\mathbf{v}\|\,|\cos\theta|.$$

Useful everywhere from graphics (shadows) to ML (closest point).

Example 1.1 (Cosine similarity)**.** In text retrieval, documents are vectors of word counts. $\cos\theta = (\mathbf{a} \cdot \mathbf{b})/(\|\mathbf{a}\|\|\mathbf{b}\|)$ near 1 means similar topics — the same formula as geometry.

1.3 Lines and Planes

A line is a point plus a direction. A plane is a point plus two independent directions.

$$\text{Line: } \mathbf{x} = \mathbf{p} + t\mathbf{d}, \quad t \in \mathbb{R}, \tag{1.1}$$

$$\text{Plane in } \mathbb{R}^3\text{: } \mathbf{x} = \mathbf{p} + s\mathbf{d}_1 + t\mathbf{d}_2, \quad s, t \in \mathbb{R}, \tag{1.2}$$

$$\text{Plane (normal form): } \mathbf{n} \cdot (\mathbf{x} - \mathbf{p}) = 0 \iff \mathbf{n} \cdot \mathbf{x} = d. \tag{1.3}$$

Here $\mathbf{d}$ is a direction vector and $\mathbf{n}$ is a *normal* (perpendicular) vector. In $\mathbb{R}^3$, if $\mathbf{d}_1, \mathbf{d}_2$ span the plane then $\mathbf{n} = \mathbf{d}_1 \times \mathbf{d}_2$ (cross product).

Example 1.2 (Distance from point to line)**.** Let $L : \mathbf{p} + t\mathbf{d}$. For $\mathbf{q}$, let $\mathbf{w} = \mathbf{q} - \mathbf{p}$. Then

$$\operatorname{dist}(\mathbf{q}, L) = \|\mathbf{w} - \operatorname{proj}_{\mathbf{d}} \mathbf{w}\| = \frac{\|\mathbf{w} \times \mathbf{d}\|}{\|\mathbf{d}\|} \quad (\text{in } \mathbb{R}^3).$$

Subtract the along-line shadow; what remains is the perpendicular distance.

Example 1.3 (Python — vectors as arrays)**.**

```python
import numpy as np
u = np.array([3.0, 1.0])
v = np.array([1.0, 2.0])
dot = u @ v                         # 5.0
norm_u = np.linalg.norm(u)          # sqrt(10)
cos_theta = dot/(norm_u*np.linalg.norm(v))
proj = (dot/(u@u))*u                # projection of v onto u? careful: swap
proj_v_on_u = (v@u)/(u@u) * u
print(cos_theta, proj_v_on_u)
```

1.4 Cross Product and Volume

In $\mathbb{R}^3$, the *cross product* $\mathbf{u} \times \mathbf{v}$ is the vector orthogonal to both $\mathbf{u}, \mathbf{v}$ with magnitude $\|\mathbf{u}\|\|\mathbf{v}\| \sin\theta$ (area of the parallelogram) and direction by the right-hand rule.

$$\mathbf{u} \times \mathbf{v} = \begin{bmatrix} u_2 v_3 - u_3 v_2 \\ u_3 v_1 - u_1 v_3 \\ u_1 v_2 - u_2 v_1 \end{bmatrix}, \quad \|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\|\|\mathbf{v}\| \sin\theta.$$

Properties: $\mathbf{u} \times \mathbf{v} = -(\mathbf{v} \times \mathbf{u})$, $\mathbf{u} \times \mathbf{u} = \mathbf{0}$, bilinear. The scalar triple product $\mathbf{u} \cdot (\mathbf{v} \times \mathbf{w}) = \det[\mathbf{u}\ \mathbf{v}\ \mathbf{w}]$ is the signed volume.

Example 1.4 (Normal to a plane)**.** Points $P(1,0,0), Q(0,1,0), R(0,0,1)$: $\overrightarrow{PQ} = [-1,1,0]^T$, $\overrightarrow{PR} = [-1,0,1]^T$, so $\mathbf{n} = \overrightarrow{PQ} \times \overrightarrow{PR} = [1,1,1]^T$. Plane: $x + y + z = 1$, matching $\mathbf{n} \cdot (\mathbf{x} - P) = 0$.

1.5 Inequalities and Geometry

Two inequalities govern all distance arguments:

Theorem 1.2 (Triangle and Cauchy–Schwarz)**.** For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$: (i) $\|\mathbf{u} + \mathbf{v}\| \le \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangle), equality when $\mathbf{u}, \mathbf{v}$ same direction; (ii) $|\mathbf{u} \cdot \mathbf{v}| \le \|\mathbf{u}\|\|\mathbf{v}\|$ with equality iff dependent.

Triangle follows from $\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + 2\mathbf{u} \cdot \mathbf{v} + \|\mathbf{v}\|^2 \le (\|\mathbf{u}\| + \|\mathbf{v}\|)^2$ via Cauchy–Schwarz. These justify calling $\|\cdot\|$ a norm and $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|$ a metric.

1.6 Chapter Notes

Vectors go back to Grassmann and Gibbs; modern computing treats them as arrays. The dot product as $\|\mathbf{u}\|\|\mathbf{v}\| \cos\theta$ unifies algebra and geometry — the bridge used in graphics, robotics, and embeddings.

1.7 Exercises

E1.1 **Arithmetic.** Let $\mathbf{u} = [2, -1, 3]^T$, $\mathbf{v} = [0, 2, -1]^T$. Compute $\mathbf{u} + \mathbf{v}$, $3\mathbf{u} - 2\mathbf{v}$, $\mathbf{u} \cdot \mathbf{v}$, $\|\mathbf{u}\|$, and the angle θ between $\mathbf{u}$ and $\mathbf{v}$ (to 1°).

E1.2 **Orthogonality.** Find all c such that $[c, 2, 1]^T$ is orthogonal to $[1, c, -2]^T$. Give a geometric interpretation of your answer.

E1.3 **Line and distance.** Let $L : \mathbf{p} = [1, 0, 2]^T$, $\mathbf{d} = [1, 1, 1]^T$. Write the parametric equation, find the point on L closest to $\mathbf{q} = [2, 3, 0]^T$, and compute $\operatorname{dist}(\mathbf{q}, L)$.

E1.4 **Projection proof.** Prove $\mathbf{v} - \operatorname{proj}_{\mathbf{u}} \mathbf{v}$ is orthogonal to $\mathbf{u}$. Use this to derive the formula for distance from a point to a plane $\mathbf{n} \cdot \mathbf{x} = d$ in $\mathbb{R}^3$.

E1.5 **Coding.** Write a Python function `gram_schmidt_2(u,v)` returning an orthonormal pair spanning the same plane as $\mathbf{u}, \mathbf{v}$ (assume independent). Test on $\mathbf{u} = [1, 1, 0]^T$, $\mathbf{v} = [1, 0, 1]^T$ and verify orthonormality numerically.

Chapter 2

Matrices

"A matrix is a spreadsheet that learned to multiply." We move from single arrows (vectors) to tables of numbers and discover the arithmetic that powers graphics, ML, and networks.

From zero: tables before theorems. A matrix is first a rectangular array — rows and columns you already know from spreadsheets. We give it addition, scaling, multiplication, and transpose, then ask what each means for vectors. No prior matrix theory assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) add, scale, multiply, and transpose matrices fluently;

(L2) interpret $A\mathbf{x}$ as a linear combination of columns and as a linear map;

(L3) recognise identity, diagonal, symmetric, and triangular matrices;

(L4) compute with block matrices and outer products;

(L5) implement matrix operations in NumPy and reason about cost.

2.1 Anatomy of a Matrix

Picture a spreadsheet: m rows, n columns, an entry in each cell.

Definition 2.1 (Matrix)**.** An $m \times n$ *matrix* over $\mathbb{R}$ is an array $A = [a_{ij}]$ with $i = 1..m$, $j = 1..n$. We write $A \in \mathbb{R}^{m \times n}$. A column vector is $m \times 1$; a row vector $1 \times n$; a scalar 1×1.

Addition and scalar multiplication are entrywise (same shape required). The zero matrix $0_{m \times n}$ and negatives are as for vectors.

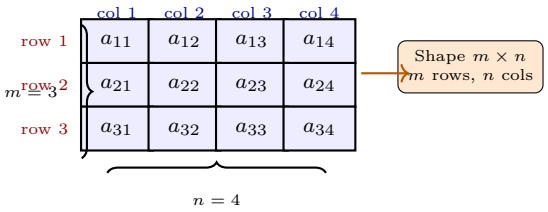

Figure 2.1: An $m \times n$ matrix as a table. Row i, column j holds a_{ij}.

2.2 Matrix Multiplication

This is the interesting one. Not entrywise — it composes linear actions.

Definition 2.2 (Product)**.** If $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, then $C = AB \in \mathbb{R}^{m \times p}$ with

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj} \quad \text{(row } i \text{ of } A \text{ dot column } j \text{ of } B\text{)}.$$

The inner dimensions must match ($n = n$); the outer give the shape.

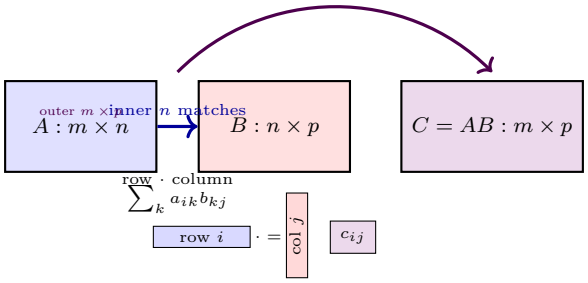

Figure 2.2: Matrix product: inner dimensions agree; entry c_{ij} is the dot product of row i of A with column j of B.

Crucial facts: $AB \neq BA$ in general; $A(BC) = (AB)C$ (associative); $A(B + C) = AB + AC$; there is an identity I_n with $I_n A = A$, $AI_n = A$.

Example 2.1 (Composition viewpoint)**.** If $B : \mathbb{R}^p \to \mathbb{R}^n$ and $A : \mathbb{R}^n \to \mathbb{R}^m$ are linear maps, then AB is the composition "do B, then A." Order matters — just as putting on socks then shoes $\neq$ shoes then socks. That is why $AB \neq BA$.

2.2.1 Four Ways to See $A\mathbf{x}$

For $A \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$:

(i) **Row dot:** $(A\mathbf{x})_i = \text{row}_i(A) \cdot \mathbf{x}$.

(ii) **Column combination:** $A\mathbf{x} = x_1 \mathbf{a}_1 + \cdots + x_n \mathbf{a}_n$ where $\mathbf{a}_j$ is column j of A. So $A\mathbf{x}$ lives in the span of the columns.

(iii) **Linear map:** $\mathbf{x} \mapsto A\mathbf{x}$ is linear: $A(c\mathbf{x} + d\mathbf{y}) = cA\mathbf{x} + dA\mathbf{y}$.

(iv) **Outer sum:** $AB = \sum_k \mathbf{a}_k \mathbf{b}_k^T$ (sum of column-times-row rank-one pieces).

View (ii) is the most useful: the columns tell you where the basis lands.

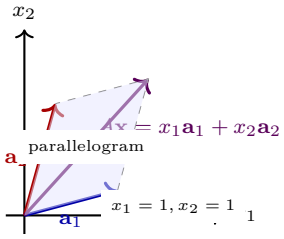

Figure 2.3: $A\mathbf{x}$ as a linear combination of the columns $\mathbf{a}_1, \mathbf{a}_2$ of A. The set of all $A\mathbf{x}$ is the column span.

2.3 Transpose and Special Matrices

Definition 2.3 (Transpose)**.** $(A^T)_{ij} = A_{ji}$ — flip across the diagonal. $(AB)^T = B^T A^T$, $(A^T)^T = A$.

A is *symmetric* if $A^T = A$ (square, $a_{ij} = a_{ji}$); *diagonal* if $a_{ij} = 0$ for $i \neq j$; *upper triangular* if $a_{ij} = 0$ for $i > j$; *identity* $I_n = \mathrm{diag}(1, \ldots, 1)$.

Trace: $\mathrm{tr}(A) = \sum_i a_{ii}$; $\mathrm{tr}(AB) = \mathrm{tr}(BA)$.

Example 2.2 (Outer product)**.** For $\mathbf{u} \in \mathbb{R}^m$, $\mathbf{v} \in \mathbb{R}^n$, the outer product $\mathbf{u}\mathbf{v}^T \in \mathbb{R}^{m \times n}$ has rank one. Inner product $\mathbf{u}^T\mathbf{v}$ is 1×1 (a scalar). This distinction matters in ML: $\mathbf{x}\mathbf{x}^T$ is a covariance building block.

Example 2.3 (Python — matrix arithmetic)**.**

```python
import numpy as np
A = np.array([[1,2,3],[4,5,6]], dtype=float)  # 2x3
B = np.array([[1,0],[0,1],[1,1]], dtype=float)  # 3x2
C = A @ B             # 2x2 matrix product
print(C)              # [[4,5],[10,11]]
print(A.T.shape)      # (3,2) -- transpose flips shape
# Column view: A @ x = x[0]*col0 + x[1]*col1 + x[2]*col2
x = np.array([1, -1, 2])
print(A @ x, 1*A[:,0] + -1*A[:,1] + 2*A[:,2])
```

2.4 Matrix Inverses

$A \in \mathbb{R}^{n \times n}$ is *invertible* (nonsingular) if there exists A^{-1} with $AA^{-1} = A^{-1}A = I_n$. Then $A\mathbf{x} = \mathbf{b}$ has unique solution $\mathbf{x} = A^{-1}\mathbf{b}$, and A represents a bijective linear map. Not all square matrices are invertible: $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ squashes $\mathbb{R}^2$ onto a line.

Theorem 2.1 (Invertibility checklist)**.** For $A \in \mathbb{R}^{n \times n}$, TFAE: (i) A invertible; (ii) $\mathcal{N}(A) = \{\mathbf{0}\}$; (iii) $\mathcal{C}(A) = \mathbb{R}^n$; (iv) $\mathrm{rank}\, A = n$; (v) $\det A \neq 0$ (Ch. 7); (vi) columns (equivalently rows) are independent.

2×2 inverse: $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ when $ad - bc \neq 0$. For larger n, elimination on $[A \mid I]$ yields $[I \mid A^{-1}]$.

2.5 Linear Maps and Matrix Representation

A function $T : \mathbb{R}^n \to \mathbb{R}^m$ is *linear* if $T(c\mathbf{u} + d\mathbf{v}) = cT(\mathbf{u}) + dT(\mathbf{v})$. Every such T is $\mathbf{x} \mapsto A\mathbf{x}$ for a unique A whose columns are $T(\mathbf{e}_j)$ ($\mathbf{e}_j$ standard basis). Composition $S \circ T$ corresponds to product BA.

Example 2.4 (Rotation in $\mathbb{R}^2$)**.** Rotation by θ counter-clockwise: $R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$. Then $R_\theta R_\phi = R_{\theta+\phi}$ and $R_\theta^T = R_{-\theta} = R_\theta^{-1}$ (orthogonal matrix).

Remark 2.1 (Why $AB \neq BA$ matters in code). In NumPy, `A @ B` and `B @ A` may even have different shapes. Computationally, $(AB)\mathbf{x} = A(B\mathbf{x})$ suggests parenthesising to minimise flops: for $A \in \mathbb{R}^{10\times100}$, $B \in \mathbb{R}^{100\times10}$, $\mathbf{x} \in \mathbb{R}^{10}$, compute $B\mathbf{x}$ first (100×10 by $10 \to O(1000)$) then $A(\cdot)$ rather than forming AB (10×10 costs $O(10 \cdot 100 \cdot 10) = 10^4$). Order and bracketing affect both meaning and cost.

Example 2.5 (Sparse matrices)**.** Many computing matrices are sparse (mostly zeros): adjacency of a graph, finite differences. Storing only nonzeros and multiplying by iterating over nonzeros saves $O(n^2)$ to $O(\text{nnz})$. `scipy.sparse` does this; dense `@` would waste time on zeros.

Example 2.6 (Hadamard vs. matrix product)**.** Entrywise (Hadamard) $A \circ B = [a_{ij}b_{ij}]$ is sometimes written $A * B$ in code (e.g. NumPy `A*B` for arrays). It is commutative and $O(mn)$, but it is *not* the matrix product AB. Mixing them is a common bug: check whether you need `A @ B` (composition) or `A * B` (masking/scaling).

Remark 2.2 (Transpose and symmetry in computing). Storing A^T is often free (stride trick, not a copy) in NumPy; `A.T` is a view. Symmetry $A^T = A$ halves storage; triangular storage saves half the entries. Numerical libraries exploit this: Cholesky for symmetric positive-definite A is $2\times$ faster than LU.

2.6 Chapter Notes

Matrix multiplication as composition explains non-commutativity and associativity at once. The column picture anticipates column space and rank (Ch. 5) and the $A = LU$ factorisation (Ch. 3). For cost, $m \times n$ times $n \times p$ is $O(mnp)$ naively; Strassen (Ch. 3 notes) does better.

2.7 Exercises

E2.1 **Product by hand.** Let $A = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & 2 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 3 & 1 \end{bmatrix}$. Compute AB and BA (if defined). Verify $(AB)^T = B^T A^T$ numerically.

E2.2 **Column view.** Write $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ and $\mathbf{x} = [2, -1]^T$. Compute $A\mathbf{x}$ both as row-dots and as a combination of columns. What is the set $\{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^2\}$ geometrically?

E2.3 **Algebra.** Prove: if A, B symmetric and $AB = BA$ then AB is symmetric. Give a 2×2 counterexample when they do not commute.

E2.4 **Block multiply.** Let $M = \begin{bmatrix} I & A \\ 0 & I \end{bmatrix}$ where A is $n \times n$. Find M^2 and M^{-1} in block form and verify by block multiplication.

E2.5 **Coding.** Benchmark `A @ B` for $n \times n$ random matrices ($n = 100, 500, 1000$) with NumPy. Plot time vs. n on a log–log scale and estimate the empirical exponent. Why is NumPy far faster than triple loops?

Chapter 3

Linear Systems and Gaussian Elimination

"Elimination is just organised subtraction." A system of equations is a puzzle of balances; Gaussian elimination solves it by the same moves you would use on paper — swap, scale, subtract — written as row operations on a matrix.

From zero: no systems background needed. We start with two lines in the plane, see the three things that can happen (one point, no point, a whole line), then grow to $m \times n$ and build an algorithm that handles them all.

Learning Outcomes

After this chapter you will be able to:

(L1) write a linear system as $A\mathbf{x} = \mathbf{b}$ and as an augmented matrix $[A \mid \mathbf{b}]$;

(L2) carry out Gaussian elimination to row-echelon and Gauss–Jordan forms;

(L3) read off existence, uniqueness, and parametric solution sets;

(L4) relate pivots, free variables, and rank to the geometry;

(L5) factor $A = LU$ and use it to solve efficiently.

3.1 Systems as Matrices

A *linear equation* is $a_1x_1 + \cdots + a_nx_n = b$. A *system* of m such equations is

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{m\times n},\ \mathbf{x} \in \mathbb{R}^n,\ \mathbf{b} \in \mathbb{R}^m.$$

The *augmented matrix* $[A \mid \mathbf{b}]$ keeps all the data in one table.

For $n > 2$ we cannot draw, but algebra does not care: the same three outcomes occur.

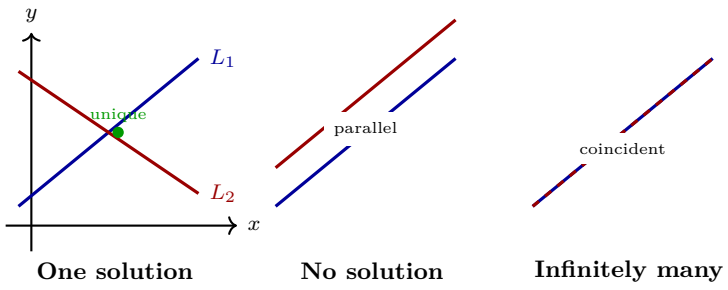

Figure 3.1: Three geometries for two equations in two unknowns. With more variables the same trichotomy holds.

3.2 Row Operations and Echelon Form

Elimination uses three *elementary row operations* on $[A \mid \mathbf{b}]$, each reversible and preserving the solution set:

(R1) Swap two rows: $R_i \leftrightarrow R_j$.

(R2) Scale a row: $R_i \leftarrow cR_i$, $c \neq 0$.

(R3) Add a multiple: $R_i \leftarrow R_i + cR_j$, $i \neq j$.

Definition 3.1 (Row-echelon form (REF))**.** A matrix is in REF if (i) all zero rows are at the bottom, (ii) each leading entry (pivot) is to the right of the one above, (iii) pivots may be any nonzero. In *reduced* REF (RREF), each pivot is 1 and is the only nonzero in its column.

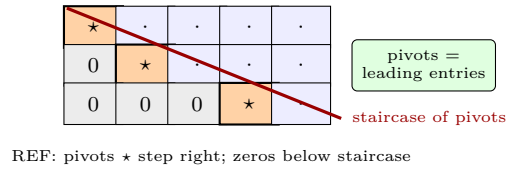

Figure 3.2: Shape of a row-echelon matrix. The staircase separates the pivot columns from free columns.

Gaussian elimination: work column by column, pick a pivot, clear below with (R3), move to next row/column. *Gauss–Jordan* continues upward to clear above pivots, reaching RREF.

Example 3.1 (Elimination by hand)**.** Solve

$$\begin{cases} x + 2y + z = 4 \\ 2x + 5y + z = 7 \\ x + y - z = 0 \end{cases} \quad \left[\begin{array}{ccc|c} 1 & 2 & 1 & 4 \\ 2 & 5 & 1 & 7 \\ 1 & 1 & -1 & 0 \end{array}\right] \xrightarrow{R_2-2R_1,\ R_3-R_1} \left[\begin{array}{ccc|c} 1 & 2 & 1 & 4 \\ 0 & 1 & -1 & -1 \\ 0 & -1 & -2 & -4 \end{array}\right] \xrightarrow{R_3+R_2} \left[\begin{array}{ccc|c} 1 & 2 & 1 & 4 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & -3 & -5 \end{array}\right].$$

Back substitution: $z = 5/3$, $y = -1 + z = 2/3$, $x = 4 - 2y - z = 1$. Unique solution $\mathbf{x} = [1, 2/3, 5/3]^T$. If a row became $[0\ 0\ 0 \mid 1]$, inconsistency — no solution.

3.3 Solution Sets: Pivots vs. Free Variables

Each column with a pivot is a *pivot column*; the rest are *free*. Free variables become parameters.

Theorem 3.1 (Existence and uniqueness)**.** $A\mathbf{x} = \mathbf{b}$ has a solution iff $\operatorname{rank}(A) = \operatorname{rank}([A \mid \mathbf{b}])$ (no pivot in the augmented column). It is unique iff every variable column is a pivot column (no free variables).

When free variables exist, the solution is an affine subspace:

$$\mathbf{x} = \mathbf{p} + t_1\mathbf{v}_1 + \cdots + t_k\mathbf{v}_k,$$

$\mathbf{p}$ a particular solution, $\{\mathbf{v}_i\}$ span the homogeneous solutions $A\mathbf{x} = \mathbf{0}$ (the null space, Ch. 4).

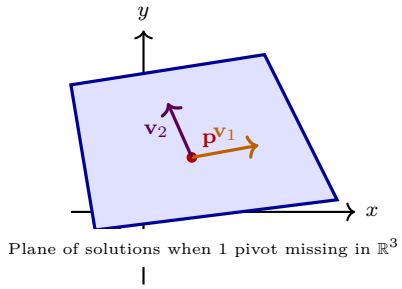

Figure 3.3: With free variables, solutions form $\mathbf{p} + \operatorname{span}\{\mathbf{v}_i\}$ — a point, line, plane, or hyperplane.

3.4 LU Factorisation

Elimination records its multipliers in L (unit lower-triangular) so that $A = LU$ with U the REF. Then $A\mathbf{x} = \mathbf{b}$ becomes $L\mathbf{y} = \mathbf{b}$ (forward substitution) then $U\mathbf{x} = \mathbf{y}$ (back substitution) — $O(n^2)$ per new $\mathbf{b}$ after one $O(n^3)$ factorisation. With row swaps, $PA = LU$.

Example 3.2 (Python — elimination and LU)**.**

```python
import numpy as np
from scipy.linalg import lu
A = np.array([[1.,2.,1.],[2.,5.,1.],[1.,1.,-1.]])
b = np.array([4.,7.,0.])
x = np.linalg.solve(A, b)
print(x)  # [1. 0.666... 1.666...]
P, L, U = lu(A)
print(np.allclose(P@L@U, A))  # True
```

If `np.linalg.matrix_rank(A)` $< n$, expect either no or infinitely many solutions — check augmented rank.

3.5 Rank and Its Geometry

The number of pivots $r = \operatorname{rank}(A)$ counts independent equations (or independent columns). Full row rank ($r = m$) means $A\mathbf{x} = \mathbf{b}$ solvable for every $\mathbf{b}$; full column rank ($r = n$) means at most one solution for each $\mathbf{b}$. When $r < \min(m, n)$, both existence and uniqueness can fail depending on $\mathbf{b}$.

Theorem 3.2 (Rank of products and sums)**.** $\operatorname{rank}(AB) \le \min(\operatorname{rank} A, \operatorname{rank} B)$; $\operatorname{rank}(A+B) \le \operatorname{rank} A + \operatorname{rank} B$. Row operations preserve rank (they are left-multiplication by invertible E).

3.6 Computational Cost and Pivoting

Elimination on $n \times n$ costs $\sim \frac{2}{3}n^3$ flops (floating-point ops); forward/back substitution $O(n^2)$. Partial pivoting (swap to bring largest $|a_{ik}|$ to pivot) avoids division by tiny numbers and controls error growth (factor $\leq 2^{n-1}$ worst-case, tiny in practice). Complete pivoting (row+column swaps) is more stable but costlier. Libraries (LAPACK) use blocked LU for cache efficiency.

Example 3.3 (When pivoting matters)**.** $A = \begin{bmatrix} 10^{-12} & 1 \\ 1 & 1 \end{bmatrix}$ without pivoting: first pivot 10^{-12} causes catastrophic cancellation in $R_2 \leftarrow R_2 - 10^{12}R_1$. Swapping rows first gives pivot 1, stable elimination.

Remark 3.1 (Geometric reading of REF). Each pivot column corresponds to a direction that is constrained; each free column is a degree of freedom. So rank A is the dimension of the column space (how many independent constraints), and $n - \operatorname{rank} A$ is the dimension of the solution line/plane when $\mathbf{b}$ is consistent. This foreshadows Ch. 5's rank–nullity.

Example 3.4 (No solution vs. infinitely many)**.** $A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$, $\mathbf{b}_1 = [1, 3]^T$: augmented REF $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$ — pivot in augmented column, no solution (parallel lines). $\mathbf{b}_2 = [1, 2]^T$: $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ — free $x_2 = t$, solutions $[1 - t, t]^T$ (coincident lines).

3.7 Chapter Notes

Gauss described elimination circa 1809 (least squares); Jordan pushed to RREF. LU is how libraries actually solve $A\mathbf{x} = \mathbf{b}$; QR (Ch. 6) is the stable alternative for least squares. Pivoting (partial: largest in column) controls round-off.

3.8 Exercises

E3.1 **REF by hand.** Reduce $\begin{bmatrix} 1 & 2 & 3 & 9 \\ 2 & 4 & 5 & 13 \\ 1 & 0 & -1 & -1 \end{bmatrix}$ to REF and RREF. Identify pivots and free variables; write all solutions.

E3.2 **Trichotomy.** Give matrices A and vectors $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ (2×2) exhibiting the three cases: unique solution, no solution, infinitely many. Explain via ranks of A and $[A \mid \mathbf{b}]$.

E3.3 **Homogeneous.** Show the solution set of $A\mathbf{x} = \mathbf{0}$ is closed under addition and scaling. Find a basis for it when $A = \begin{bmatrix} 1 & 2 & -1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}$.

E3.4 **LU.** Compute L and U (no pivoting) for $A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 5 & 2 \\ 2 & 1 & -1 \end{bmatrix}$ by recording multipliers. Verify $A = LU$ and solve $A\mathbf{x} = [1, 2, 0]^T$ via L, U.

E3.5 **Coding.** Write `rref(A)` (Gauss–Jordan, partial pivoting, tolerance 10^{-10}) returning RREF and pivot columns. Test on random 3×5 matrices and compare pivot columns with `np.linalg.matrix_rank`.

Chapter 4

Vector Spaces

"A vector space is any place where arrows still make sense." We abstract $\mathbb{R}^n$ to uncover spaces of matrices, polynomials, signals, and images — all governed by the same eight rules.

From zero: axioms after examples. We have lived in $\mathbb{R}^n$ for three chapters. Now we ask what makes it tick, write the rules down, and test which other collections obey them. Every axiom is motivated by a picture or a failure.

Learning Outcomes

After this chapter you will be able to:

(L1) state the eight vector-space axioms and test a set against them;

(L2) recognise subspaces and verify them via the subspace test;

(L3) describe column space, null space, row space, and their relations;

(L4) decide membership in a span and express vectors as combinations;

(L5) connect subspaces to solutions of $A\mathbf{x} = \mathbf{b}$ and $A\mathbf{x} = \mathbf{0}$.

4.1 Why Abstract?

In $\mathbb{R}^n$ we add arrows tip-to-tail and stretch them. But we also add *polynomials* $(p+q)(x) = p(x)+q(x)$, *matrices*, *audio signals* (add waveforms, scale volume), and *RGB images*. Each supports the same two operations. Rather than prove everything four times, we isolate the common rules.

4.2 The Axioms

A *vector space* over $\mathbb{R}$ is a set V with addition $+$ and scalar multiplication $\cdot$ satisfying, for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$, $c, d \in \mathbb{R}$:

(V1) $\mathbf{u} + \mathbf{v} \in V$ (closure under $+$), $\quad c\mathbf{u} \in V$ (closure under scaling);

(V2) $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$; $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$;

(V3) there exists $\mathbf{0} \in V$ with $\mathbf{v} + \mathbf{0} = \mathbf{v}$;

(V4) for each $\mathbf{v}$ there is $-\mathbf{v}$ with $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$;

(V5) $c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v}$; $\;(c + d)\mathbf{v} = c\mathbf{v} + d\mathbf{v}$;

(V6) $c(d\mathbf{v}) = (cd)\mathbf{v}$; $\;1\mathbf{v} = \mathbf{v}$.

Elements of V are called *vectors* even when they look like matrices or functions.

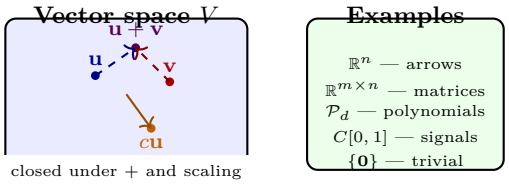

Figure 4.1: Left: closure means sums and scalings never leave V. Right: diverse examples obeying the same axioms.

Example 4.1 (Polynomials)**.** $\mathcal{P}_2 = \{a_0 + a_1 x + a_2 x^2 : a_i \in \mathbb{R}\}$ is a vector space of dimension 3 (basis $1, x, x^2$). The set $\{p : p(0) = 1\}$ is *not* a subspace (fails $\mathbf{0}$ and closure — sum has $p(0) = 2$).

Example 4.2 (Non-example)**.** The first quadrant $\{(x, y) : x \geq 0, y \geq 0\} \subset \mathbb{R}^2$ is not a vector space: $(1, 1)$ is in it but $-(1, 1) = (-1, -1)$ is not (fails (V4) and scaling by -1).

4.3 Subspaces

Definition 4.1 (Subspace)**.** $W \subseteq V$ is a *subspace* if it is itself a vector space under the same $+$ and $\cdot$. Equivalently (subspace test):

$$W \neq \emptyset, \quad \mathbf{u}, \mathbf{v} \in W \Rightarrow \mathbf{u} + \mathbf{v} \in W, \quad c \in \mathbb{R}, \mathbf{u} \in W \Rightarrow c\mathbf{u} \in W.$$

In particular $\mathbf{0} \in W$ always.

4.4 Four Fundamental Subspaces of a Matrix

For $A \in \mathbb{R}^{m \times n}$:

- **Column space** $\mathcal{C}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$ — all $\mathbf{b}$ for which $A\mathbf{x} = \mathbf{b}$ is solvable. Span of the columns.

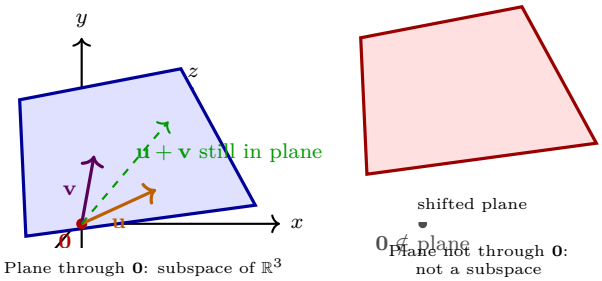

Figure 4.2: Left: a subspace must contain $\mathbf{0}$ and be closed (plane through origin). Right: a translate fails the test.

- **Null space** $\mathcal{N}(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\} \subseteq \mathbb{R}^n$ — homogeneous solutions (Ch. 3). Always a subspace.
- **Row space** $\mathcal{C}(A^T) \subseteq \mathbb{R}^n$ — span of the rows.
- **Left null space** $\mathcal{N}(A^T) \subseteq \mathbb{R}^m$.

$\mathcal{C}(A)$ and $\mathcal{N}(A^T)$ live in $\mathbb{R}^m$; $\mathcal{N}(A)$ and $\mathcal{C}(A^T)$ in $\mathbb{R}^n$. Row operations preserve $\mathcal{N}(A)$ and row space but change $\mathcal{C}(A)$ — a common pitfall.

Example 4.3 (Membership)**.** Is $\mathbf{b} = [1,2,3]^T$ in $\mathcal{C}(A)$ for $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$? Second column is $2\times$ first, so $\mathcal{C}(A) = \text{span}\{[1,2,3]^T\}$. Since $\mathbf{b} = 1 \cdot [1,2,3]^T$, yes. $\mathbf{b}' = [1,0,0]^T$ is not.

Example 4.4 (Python — null space)**.**

```python
import numpy as np
from scipy.linalg import null_space
A = np.array([[1.,2.,1.],[2.,4.,2.],[1.,1.,0.]])
print("rank", np.linalg.matrix_rank(A))
NS = null_space(A)   # orthonormal basis for N(A)
print(NS.T)          # each row is a basis vector
print(A @ NS)        # ~ 0 (up to round-off)
```

4.5 Span and Spanning Sets

For $S = \{\mathbf{v}_1, \ldots, \mathbf{v}_k\} \subseteq V$, $\text{span}(S) = \{\sum c_i \mathbf{v}_i : c_i \in \mathbb{R}\}$ is the smallest subspace containing S (intersection of all subspaces containing S). S *spans* V if $\text{span}(S) = V$. Spanning is the opposite of independence: too many vectors guarantees dependence, too few cannot span.

Theorem 4.1 (Spanning test via matrices)**.** Fix a basis to coordinatize $V \cong \mathbb{R}^n$. Then S spans V iff the matrix with columns $[\mathbf{v}_i]$ has rank n (every $\mathbf{b}$ is a combination).

4.6 Sums and Direct Sums

For subspaces $U, W \subseteq V$, $U + W = \{\mathbf{u} + \mathbf{w} : \mathbf{u} \in U, \mathbf{w} \in W\}$ is a subspace; $U \cap W$ is too. The sum is *direct*, $V = U \oplus W$, if $V = U + W$ and $U \cap W = \{\mathbf{0}\}$ — then every $\mathbf{v} \in V$ has a unique $\mathbf{u}+\mathbf{w}$ decomposition. Example: $\mathbb{R}^3 = xy\text{-plane} \oplus z\text{-axis}$.

Example 4.5 (Polynomial split)**.** $\mathcal{P}_3 = \{p : p(0) = 0\} \oplus \{\text{constants}\}$: every $p(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ uniquely as $(a_1x + a_2x^2 + a_3x^3) + a_0$. Dimensions add: $3 + 1 = 4 = \dim \mathcal{P}_3$.

4.7 Coordinates Revisited

Fix a basis $B = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ of V. The map $\mathbf{v} \mapsto [\mathbf{v}]_B \in \mathbb{R}^n$ (coordinates) is a linear isomorphism: it preserves $+$ and scaling and is bijective. So every n-dimensional real vector space is "just" $\mathbb{R}^n$ in disguise — but the disguise matters for interpretation (e.g. polynomial vs. signal).

Example 4.6 (Function space: audio signals)**.** Let $V = \{f : [0, 1] \to \mathbb{R} \text{ continuous}\}$ with $(f+g)(t) = f(t)+g(t)$, $(cf)(t) = c\,f(t)$. This is infinite-dimensional: $\{1, t, t^2, \dots\}$ are independent but never span all continuous signals (need Fourier or polynomial approximations). Subspace $W = \{f : f(0) = 0\}$ contains $\mathbf{0}$ and is closed; $W' = \{f : f(0) = 1\}$ is not.

Remark 4.1 (Why $\mathbf{0} \in W$ is non-negotiable)*.* If W is closed under scaling, $0 \cdot \mathbf{w} = \mathbf{0} \in W$ for any $\mathbf{w} \in W$ (provided $W \neq \emptyset$). So checking $\mathbf{0} \in W$ is the fastest way to kill a non-subspace: the shifted plane, the set $\det = 1$, the unit circle all fail instantly.

4.8 Linear Combinations and Spanning Revisited

A *linear combination* of $\mathbf{v}_1, \dots, \mathbf{v}_k$ is $\sum c_i\mathbf{v}_i$. The set of all such is $\text{span}\{\mathbf{v}_i\}$, already a subspace. To test if $\mathbf{b} \in \text{span}\{\mathbf{v}_i\}$, solve $[\mathbf{v}_1 \cdots \mathbf{v}_k]\mathbf{x} = \mathbf{b}$ — this is exactly $A\mathbf{x} = \mathbf{b}$ from Ch. 3. Geometry (Ch. 1) and algebra (Ch. 2) meet: $\mathbf{b}$ is in the span iff it is a column combination, i.e. $\mathbf{b} \in \mathcal{C}(A)$.

Example 4.7 (Polynomial coordinates)**.** In $\mathcal{P}_2$ with basis $\{1, x, x^2\}$, $p(x) = 3 - x + 2x^2$ has $[p]_B = [3, -1, 2]^T$. Addition and scaling act entrywise on coordinates, so $\mathcal{P}_2 \cong \mathbb{R}^3$ as vector spaces — but p is a function, not an arrow, reminding us the isomorphism is interpretation-dependent.

Remark 4.2 (Subspace vs. subset: a one-line test)*.* To kill a non-subspace, find one closure failure. The set $\{A : \det A = 1\}$ fails scaling ($2A$ has $\det 2^n$); the unit circle fails addition ($(1, 0) + (0, 1)$ outside). One counterexample suffices.

4.9 Chapter Notes

Abstract vector spaces (Peano, 1888; Banach, 1920s) let one theorem serve many domains. Subspaces are the "lines/planes through the origin" in any V. The four subspaces reappear in Ch. 6 as orthogonal complements.

4.10 Exercises

E4.1 **Axiom check.** Which of these are vector spaces? (a) $\{(x, y) : xy \geq 0\}$ in $\mathbb{R}^2$, (b) $\{p \in \mathcal{P}_2 : p(1) = 0\}$, (c) $\{A \in \mathbb{R}^{2\times 2} : \det A = 0\}$. Justify via the subspace test or a counterexample.

E4.2 **Span membership.** Let $A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 1 & 1 & 3 \end{bmatrix}$. Is $\mathbf{b} = [3, 2, 5]^T$ in $\mathcal{C}(A)$? Is $\mathbf{x} = [2, -1, 1]^T$ in $\mathcal{N}(A)$? Decide by solving $A\mathbf{x} = \mathbf{b}$ / $A\mathbf{x} = \mathbf{0}$.

E4.3 **Null space.** Find $\mathcal{N}(A)$ for $A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$ explicitly as $\text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$. Verify closure under addition.

E4.4 **Intersection.** Prove the intersection of two subspaces of V is a subspace. Is the union always a subspace? Give a counterexample in $\mathbb{R}^2$.

E4.5 **Coding.** Write `is_in_colspace(A,b,tol=1e-9)` using rank: $\mathbf{b} \in \mathcal{C}(A)$ iff $\text{rank}(A) = \text{rank}([A \mid \mathbf{b}])$. Test on random 4×3 matrices.

Chapter 5

Linear Independence, Basis, and Dimension

"Independence means no vector is a passenger — each pulls in a new direction." This chapter asks when a set is redundant, how to extract the essentials, and how many essentials any space needs.

From zero: redundancy before rank. We build independence from the question "can one vector be made from the others?" then define basis as a minimal spanning set and dimension as its size — proved well-defined.

Learning Outcomes

After this chapter you will be able to:

(L1) test a set for linear independence via $A\mathbf{x} = \mathbf{0}$;

(L2) extract a basis for $\mathcal{C}(A)$, $\mathcal{N}(A)$, and row space from REF;

(L3) compute rank, nullity, and apply rank–nullity;

(L4) express vectors uniquely in a basis (coordinates) and change basis;

(L5) interpret dimension as degrees of freedom in computing systems.

5.1 Linear Independence

Picture three arrows in $\mathbb{R}^2$. The third must lie in the plane of the first two, so it is a combination of them — redundant. Independence means no such redundancy.

Definition 5.1 (Independence)**.** Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k \in V$ are *linearly independent* if

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0} \implies c_1 = \cdots = c_k = 0.$$

Otherwise they are *dependent*: some nontrivial combination gives $\mathbf{0}$ (equivalently, one vector is a combination of the others).

Test: put vectors as columns of $A = [\mathbf{v}_1 \cdots \mathbf{v}_k]$; independence $\iff \mathcal{N}(A) = \{\mathbf{0}\} \iff$ every column is a pivot column in REF.

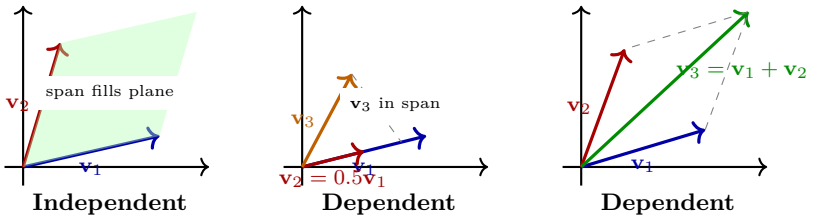

Figure 5.1: Left: independent — new direction each time. Middle/Right: dependent — one vector lies in the span of the others.

Example 5.1 (Polynomials)**.** In $\mathcal{P}_2$, $\{1,\ x,\ x^2\}$ is independent; $\{1,\ x,\ 1{+}x\}$ is dependent since $(1{+}x){-}1{-}x = 0$. The test is the same: nontrivial coefficients making the zero polynomial.

5.2 Span, Basis, Dimension

Definition 5.2 (Span and basis)**.** $\operatorname{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ is the set of all combinations $\sum c_i \mathbf{v}_i$. A *basis* of V is a set that (i) is independent and (ii) spans V. The *dimension* $\dim V$ is the number of vectors in any basis (proved well-defined below).

Bases are not unique, but size is. Coordinates are unique once a basis is fixed.

Theorem 5.1 (Dimension well-defined)**.** Any two bases of a finite-dimensional V have the same cardinality. Any independent set can be extended to a basis; any spanning set contains a basis.

Sketch. If B is a basis (n vectors) and S is independent (k vectors), each $\mathbf{s} \in S$ is a unique combination of B — the coordinate matrix is $n \times k$ with independent columns, so $k \le n$. Symmetrically $n \le k$ when both are bases. □

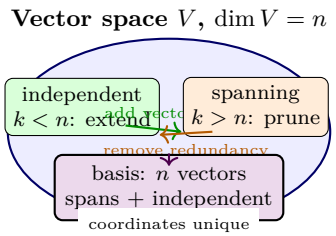

Figure 5.2: A basis is a minimal spanning set and a maximal independent set; its size is the dimension.

5.3 Bases for the Four Subspaces; Rank–Nullity

For $A \in \mathbb{R}^{m \times n}$ with REF R:

- $\mathcal{C}(A)$: basis = columns of A at pivot positions of R (not columns of R).
- $\mathcal{N}(A)$: free variables give one basis vector each (set one free = 1, rest 0, solve for pivots).
- Row space: nonzero rows of R (or R^T columns).

Let $r = \operatorname{rank}(A)$ = number of pivots = $\dim \mathcal{C}(A) = \dim$ row space.

Theorem 5.2 (Rank–nullity)**.** For $A \in \mathbb{R}^{m\times n}$,

$$\operatorname{rank}(A) + \operatorname{nullity}(A) = n, \qquad \operatorname{nullity}(A) = \dim \mathcal{N}(A).$$

Also $\dim \mathcal{N}(A^T) = m - r$. So $r \le \min(m, n)$.

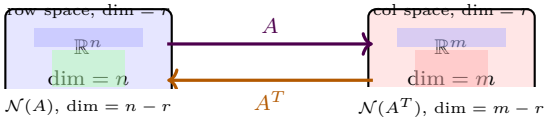

Figure 5.3: The four subspaces and their dimensions. The row and column spaces both have dimension $r = \operatorname{rank}(A)$.

Example 5.2 (Rank–nullity in action)**.** $A = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 2 & 4 & 3 & 1 \\ 1 & 2 & 2 & 1 \end{bmatrix} \xrightarrow{\text{REF}} \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$. Pivots in cols $1, 3$ so $r = 2$, nullity $= 4 - 2 = 2$. $\mathcal{C}(A)$ basis: cols $1, 3$ of A; $\mathcal{N}(A)$ basis: from x_2, x_4 free.

Example 5.3 (Coordinates and change of basis)**.** Fix basis $B = \{\mathbf{b}_1, \mathbf{b}_2\}$. Then $[\mathbf{v}]_B$ solves $[\mathbf{b}_1\ \mathbf{b}_2][\mathbf{v}]_B = \mathbf{v}$. If C is another basis, $[\mathbf{v}]_C = P_{C\leftarrow B}[\mathbf{v}]_B$ where P has columns $[\mathbf{b}_j]_C$.

```python
import numpy as np
B = np.array([[1.,1.],[0.,1.]]).T  # columns are b1,b2 -- careful with layout
# Actually B_cols = [[1,0],[1,1]] as columns
B_cols = np.array([[1.,1.],[0.,1.]])  # col j = B_cols[:,j]
v = np.array([3.,2.])
coords = np.linalg.solve(B_cols, v)
print(coords)  # [1., 2.] meaning v = 1*b1 + 2*b2
print(B_cols @ coords)  # [3., 2.] check
```

5.4 Exchange and Dimension Uniqueness

Lemma 5.1 (Steinitz exchange)**.** If $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ spans V and $\{\mathbf{u}_1, \dots, \mathbf{u}_k\}$ is independent, then $k \le n$ and the span can exchange k of the $\mathbf{v}_i$ for the $\mathbf{u}_j$ and still span.

Corollary: all bases have the same size; any independent set extends to a basis (keep adding vectors outside the current span until spanning); any spanning set prunes to a basis (drop dependent vectors one by one). This justifies calling $\dim V$ well-defined.

Example 5.4 (Extending in $\mathcal{P}_2$)**.** $\{\mathbf{v}_1 = 1 + x\}$ independent in $\mathcal{P}_2$ ($\dim 3$) extends to a basis, e.g. $\{1 + x,\ x,\ x^2\}$ or $\{1 + x,\ 1 - x,\ x^2\}$. Infinitely many bases; dimension is the invariant.

5.5 Matrix Rank = Row Rank = Column Rank

A nontrivial theorem: $\dim \mathcal{C}(A) = \dim \mathcal{C}(A^T)$ — column rank equals row rank, both $= r$. So REF pivot count is intrinsic, not an artefact. Proof sketch: show $\mathcal{N}(A)$ is orthogonal complement of row space (Ch. 6); then rank–nullity gives equality.

5.6 Change of Basis and Similarity Preview

If B and C are bases, $P_{C\leftarrow B}$ (columns $[\mathbf{b}_j]_C$) converts $[\mathbf{v}]_B$ to $[\mathbf{v}]_C$. For a linear operator $T : V \to V$, its matrices satisfy $[T]_C = P[T]_B P^{-1}$ — similarity. Determinant, trace, eigenvalues are similarity invariants (Ch. 7–8). This is why diagonalization $A = PDP^{-1}$ is a change to the eigenbasis where T acts diagonally.

5.7 Chapter Notes

Grassmann (1844) first framed dimension this way. Rank–nullity is the "conservation of dimensions" for $A : \mathbb{R}^n \to \mathbb{R}^m$. Computing rank via SVD (Ch. 6/8 notes) is numerically stable; REF rank is exact-arithmetic.

5.8 Exercises

E5.1 **Test.** Are $\mathbf{v}_1 = [1, 0, 1]^T$, $\mathbf{v}_2 = [0, 1, 1]^T$, $\mathbf{v}_3 = [1, 1, 2]^T$ independent? If dependent, write one as a combination of the others.

E5.2 **Bases from REF.** For $A = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 1 & 2 & 1 & 3 \end{bmatrix}$, find bases for $\mathcal{C}(A)$, $\mathcal{N}(A)$, and row space. Verify rank+nullity= n and $r \leq \min(m, n)$.

E5.3 **Coordinates.** Let $B = \{[1, 1]^T, [1, -1]^T\}$ be a basis of $\mathbb{R}^2$. Find $[\mathbf{v}]_B$ for $\mathbf{v} = [3, 1]^T$. If C is the standard basis, write $P_{C\leftarrow B}$ and $P_{B\leftarrow C}$.

E5.4 **Dimension bound.** Prove any $k > n$ vectors in $\mathbb{R}^n$ are dependent. Deduce $\dim \mathbb{R}^n = n$ via the standard basis.

E5.5 **Coding.** Write `col_basis(A)` returning indices of pivot columns (tolerance 10^{-10}) and a basis for $\mathcal{N}(A)$ via SVD or REF. Compare with `scipy.linalg.null_space` on random 5×7 matrices.

Chapter 6

Orthogonality and Least Squares

"Orthogonality is geometry's way of saying 'no shadow'." When a system $A\mathbf{x} = \mathbf{b}$ has no solution, the closest we can do is the shadow of $\mathbf{b}$ on the column space — that shadow solves the least-squares problem.

From zero: right angles before residuals. We generalise $\mathbf{u} \cdot \mathbf{v} = 0$ from $\mathbb{R}^2$ to $\mathbb{R}^n$, build orthogonal projections, then solve $A\mathbf{x} \approx \mathbf{b}$ when equality is impossible — the workhorse of regression and data fitting.

Learning Outcomes

After this chapter you will be able to:

(L1) test orthogonality and orthonormality of sets;

(L2) compute orthogonal projections onto subspaces;

(L3) run Gram–Schmidt to build orthonormal bases and QR;

(L4) formulate and solve least-squares via normal equations and QR;

(L5) apply least squares to line fitting and explain residuals.

6.1 Orthogonality in $\mathbb{R}^n$

Recall $\mathbf{u} \perp \mathbf{v} \iff \mathbf{u}\cdot\mathbf{v} = 0$ (Ch. 1). A set $\{\mathbf{q}_1, \ldots, \mathbf{q}_k\}$ is *orthogonal* if each pair is orthogonal; *orthonormal* if in addition $\|\mathbf{q}_i\| = 1$. Orthonormal sets are automatically independent — no one can be built from perpendicular directions.

If $Q = [\mathbf{q}_1 \cdots \mathbf{q}_k]$ has orthonormal columns, then $Q^TQ = I_k$ — columns are perfectly conditioned; Q preserves lengths: $\|Q\mathbf{x}\| = \|\mathbf{x}\|$.

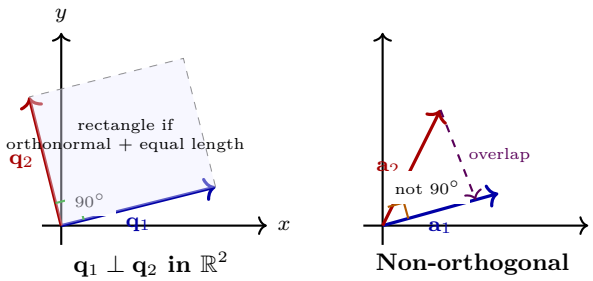

Figure 6.1: Left: orthogonal vectors meet at 90°; orthonormal adds unit length. Right: general non-orthogonal pair (Gram–Schmidt will fix this).

6.2 Projections onto Subspaces

Let $W = \operatorname{span}\{\mathbf{a}_1, \dots, \mathbf{a}_k\} \subseteq \mathbb{R}^n$. For $\mathbf{b} \in \mathbb{R}^n$, the *projection* $\hat{\mathbf{b}} = \operatorname{proj}_W \mathbf{b}$ is the unique $\mathbf{w} \in W$ closest to $\mathbf{b}$; the residual $\mathbf{b} - \hat{\mathbf{b}}$ is orthogonal to W.

When W has orthonormal basis Q,

$$\operatorname{proj}_W \mathbf{b} = QQ^T\mathbf{b} = \sum_{i=1}^{k} (\mathbf{q}_i^T \mathbf{b})\, \mathbf{q}_i.$$

In general with $A = [\mathbf{a}_1 \cdots \mathbf{a}_k]$ (full column rank),

$$\operatorname{proj}_W \mathbf{b} = A(A^TA)^{-1}A^T\mathbf{b}.$$

The matrix $P = A(A^TA)^{-1}A^T$ satisfies $P^2 = P$, $P^T = P$ — the orthogonal projector onto $\mathcal{C}(A)$.

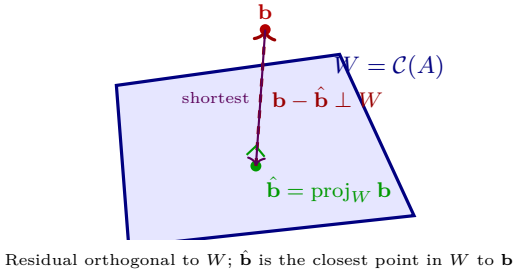

Figure 6.2: Orthogonal projection: $\hat{\mathbf{b}}$ is the closest point in W to $\mathbf{b}$; the error $\mathbf{b} - \hat{\mathbf{b}}$ is perpendicular to W.

6.3 Gram–Schmidt and QR

Gram–Schmidt turns any independent $\mathbf{a}_1, \dots, \mathbf{a}_k$ into orthonormal $\mathbf{q}_1, \dots, \mathbf{q}_k$ spanning the same nested subspaces:

$$\mathbf{u}_1 = \mathbf{a}_1, \quad \mathbf{q}_1 = \mathbf{u}_1/\|\mathbf{u}_1\|, \tag{6.1}$$

$$\mathbf{u}_2 = \mathbf{a}_2 - (\mathbf{q}_1^T\mathbf{a}_2)\mathbf{q}_1, \quad \mathbf{q}_2 = \mathbf{u}_2/\|\mathbf{u}_2\|, \tag{6.2}$$

$$\vdots \tag{6.3}$$

$$\mathbf{u}_k = \mathbf{a}_k - \sum_{i<k}(\mathbf{q}_i^T\mathbf{a}_k)\mathbf{q}_i. \tag{6.4}$$

Each step subtracts the shadows already accounted for. With $A = QR$ (Q orthonormal columns, R upper-triangular), $R = Q^TA$ records the coefficients.

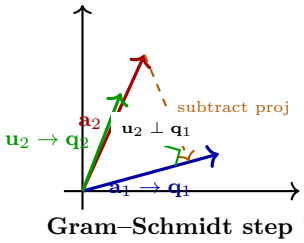

Figure 6.3: Gram–Schmidt: subtract from $\mathbf{a}_2$ its projection on $\mathbf{q}_1$; what remains is orthogonal to $\mathbf{q}_1$. Normalise to get $\mathbf{q}_2$.

Example 6.1 (QR by hand (2×2)). $A = \begin{bmatrix}1 & 1\\ 1 & 0\\ 0 & 1\end{bmatrix}$ (3×2): $\mathbf{q}_1 = [1,1,0]^T/\sqrt{2}$, $\mathbf{u}_2 = [1,0,1]^T - (\mathbf{q}_1^T[1,0,1]^T)\mathbf{q}_1 = [1/2, -1/2, 1]^T$, $\mathbf{q}_2 = \mathbf{u}_2/\|\mathbf{u}_2\|$. Then $R = Q^TA$ is 2×2 upper-triangular.

6.4 Least Squares

When $A\mathbf{x} = \mathbf{b}$ has no solution ($\mathbf{b} \notin \mathcal{C}(A)$), minimise the squared error:

$$\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|^2.$$

The minimiser $\hat{\mathbf{x}}$ makes $A\hat{\mathbf{x}} = \text{proj}_{\mathcal{C}(A)}\,\mathbf{b}$, so the residual is orthogonal to $\mathcal{C}(A)$: $A^T(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0}$.

Theorem 6.1 (Normal equations). $\hat{\mathbf{x}}$ is a least-squares solution iff

$$A^TA\hat{\mathbf{x}} = A^T\mathbf{b}.$$

If A has independent columns, A^TA is invertible and $\hat{\mathbf{x}} = (A^TA)^{-1}A^T\mathbf{b}$ is unique. With $A = QR$, $\hat{\mathbf{x}} = R^{-1}Q^T\mathbf{b}$ (more stable).

Example 6.2 (Line fitting). Fit $y = c + dt$ through (t_i, y_i): $A = \begin{bmatrix}1 & t_1\\ \vdots & \vdots\\ 1 & t_m\end{bmatrix}$, $\mathbf{x} = [c,d]^T$. Then $A^TA = \begin{bmatrix}m & \sum t_i\\ \sum t_i & \sum t_i^2\end{bmatrix}$, $A^T\mathbf{b} = [\sum y_i, \sum t_iy_i]^T$. Solving gives the regression line; $\|\mathbf{b} - A\hat{\mathbf{x}}\|$ is the residual.

```python
import numpy as np
t = np.array([0.,1.,2.,3.])
y = np.array([1.1,1.9,3.2,3.8])
A = np.column_stack([np.ones_like(t), t])
x_hat, residuals, rank, s = np.linalg.lstsq(A, y, rcond=None)
print(x_hat)  # [c, d] intercept + slope
# Compare with normal equations:
print(np.linalg.solve(A.T@A, A.T@y))
```

6.5 Orthogonal Complements and the Four Subspaces

For $W \subseteq \mathbb{R}^n$, $W^\perp = \{\mathbf{x} : \mathbf{x} \perp \mathbf{w}\ \forall \mathbf{w} \in W\}$ is a subspace with $\dim W + \dim W^\perp = n$ and $(W^\perp)^\perp = W$. Fundamental theorem:

$$\mathcal{C}(A)^\perp = \mathcal{N}(A^T), \qquad \mathcal{C}(A^T)^\perp = \mathcal{N}(A).$$

So $\mathbb{R}^n = \mathcal{C}(A^T) \oplus \mathcal{N}(A)$ and $\mathbb{R}^m = \mathcal{C}(A) \oplus \mathcal{N}(A^T)$ as orthogonal direct sums. The least-squares residual lives in $\mathcal{N}(A^T)$.

6.6 Best Approximation and the Projection Theorem

Theorem 6.2 (Projection theorem)**.** Let $W \subseteq \mathbb{R}^n$ be a subspace. For any $\mathbf{b}$, there is a unique $\hat{\mathbf{b}} \in W$ minimising $\|\mathbf{b} - \hat{\mathbf{b}}\|$, namely $\hat{\mathbf{b}} = \text{proj}_W \mathbf{b}$. Moreover $\mathbf{b} - \hat{\mathbf{b}} \in W^\perp$.

This underpins all of least squares: $A\hat{\mathbf{x}}$ is the best approximation to $\mathbf{b}$ from $\mathcal{C}(A)$. Error $\|\mathbf{b} - A\hat{\mathbf{x}}\|$ is the distance from $\mathbf{b}$ to the subspace.

6.7 Numerical Stability: QR vs. Normal Equations

Forming A^TA squares the condition number $\kappa(A^TA) = \kappa(A)^2$, so normal equations lose half the digits when A is ill-conditioned. QR avoids this: Q is perfectly conditioned ($\kappa(Q) = 1$), so solving $R\hat{\mathbf{x}} = Q^T\mathbf{b}$ is stable. Modified Gram–Schmidt and Householder QR are preferred; NumPy's `lstsq` uses SVD/Householder internally.

Example 6.3 (Polynomial least squares)**.** Fit a parabola $y = c_0 + c_1 t + c_2 t^2$ to m points: A is $m \times 3$ Vandermonde $[1\ t_i\ t_i^2]$. Columns become nearly dependent for large t_i (ill-conditioned); centering t_i or using orthogonal polynomials (Legendre) fixes this. `np.polyfit` does a scaled QR internally for this reason.

Remark 6.1 (When A^TA is singular)*.* If A has dependent columns, A^TA singular and normal equations have infinitely many solutions. The minimal-norm least-squares solution is $A^\dagger\mathbf{b}$ where $A^\dagger = V\Sigma^\dagger U^T$ is the pseudoinverse (SVD, Ch. 8). NumPy's `lstsq` returns this by default.

Remark 6.2 (Geometric meaning of $Q^TQ = I$)*.* Columns $\mathbf{q}_i$ orthonormal $\iff$ Q is an isometry on its column space: angles and lengths preserved. In graphics, Q is a rotation/reflection; in data science, Q from QR is the "well-conditioned" part of A. Never form Q explicitly when $m \gg n$ — Householder applies $Q^T\mathbf{b}$ implicitly in $O(mn)$.

6.8 Chapter Notes

Least squares (Legendre, Gauss, 1805–1809) predates matrices; Gauss used it to predict Ceres' orbit. QR via Gram–Schmidt or Householder reflections is the stable solver (LAPACK's `gels`); forming A^TA squares the condition number and is avoided for ill-conditioned A.

6.9 Exercises

E6.1 **Orthonormal check.** Is $Q = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$ orthogonal ($Q^TQ = I$)? What is $Q\mathbf{x}$ geometrically?

E6.2 **Gram–Schmidt.** Apply Gram–Schmidt to $\mathbf{a}_1 = [1,1,0]^T$, $\mathbf{a}_2 = [1,0,1]^T$, $\mathbf{a}_3 = [0,1,1]^T$ in $\mathbb{R}^3$. Write $A = QR$ and verify $Q^TQ = I$, $A = QR$.

E6.3 **Projection matrix.** For $A = [1,1,1]^T$ (span of all-ones vector in $\mathbb{R}^3$), compute $P = A(A^TA)^{-1}A^T$. Show $P^2 = P$, $P^T = P$, and $P\mathbf{b}$ is the mean of entries times $\mathbf{1}$.

E6.4 **Least squares.** Fit $y = c + dt$ to $(0,1), (1,2), (2,2), (3,4)$ via normal equations. Compute $\hat{\mathbf{x}}$ and $\|\mathbf{b} - A\hat{\mathbf{x}}\|$. Verify $A^T(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0}$.

E6.5 **Coding.** Generate $m = 100$ noisy points $y = 2 + 3t + \mathcal{N}(0, 0.5)$, $t \in [0, 5]$. Solve least squares via (a) normal equations and (b) `np.linalg.lstsq` (QR). Compare $\hat{\mathbf{x}}$ and residuals; which is more stable if you add an outlier?

Chapter 7

Determinants

"The determinant is the signed volume of the box the matrix builds." Zero means the box is flat — the matrix squashes space and cannot be undone.

From zero: area before algebra. We start with area of a parallelogram in $\mathbb{R}^2$ ($ad - bc$), grow to volume in $\mathbb{R}^3$, then axiomatise to $n \times n$ via row operations. Every property is first a picture.

Learning Outcomes

After this chapter you will be able to:

(L1) compute 2×2 and 3×3 determinants and interpret them as area/volume;

(L2) apply row-operation rules and multiplicativity $\det(AB) = \det A \det B$;

(L3) use $\det A \neq 0$ to test invertibility and find $\det(A^{-1})$;

(L4) expand by cofactors and relate to the adjugate;

(L5) connect det to eigenvalues and to Cramer's rule.

7.1 Area, Volume, and the 2×2 Determinant

Take columns $\mathbf{a}_1, \mathbf{a}_2 \in \mathbb{R}^2$ as edges from the origin. They span a parallelogram. Its signed area is

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.$$

Sign encodes orientation: $\det > 0$ means $\mathbf{a}_2$ is counter-clockwise from $\mathbf{a}_1$.

In $\mathbb{R}^3$, $\det[\mathbf{a}_1\ \mathbf{a}_2\ \mathbf{a}_3]$ is the signed volume of the parallelepiped. The pattern: determinant measures how the matrix scales volumes.

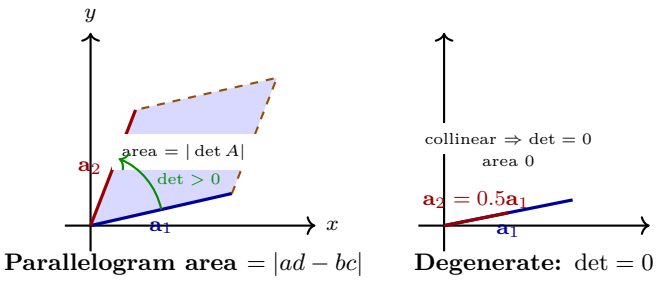

Figure 7.1: Left: $\det A$ is the signed area of the column parallelogram. Right: dependent columns give zero area — the box is flat.

7.2 Axioms and Row-Operation Rules

For $n \times n$, det is the unique function $\mathbb{R}^{n\times n} \to \mathbb{R}$ that is (i) multilinear in rows, (ii) alternating (zero if two rows equal), (iii) $\det I_n = 1$. From this:

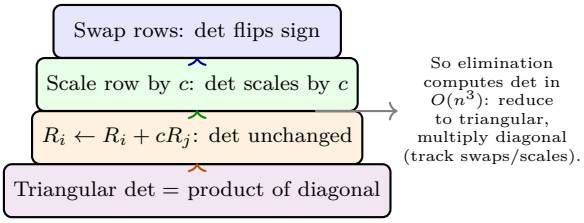

Figure 7.2: How row operations affect the determinant. Elimination to triangular form is the practical algorithm.

Consequences:

- $\det(AB) = \det A \cdot \det B$, $\det(A^T) = \det A$.
- A invertible $\iff$ $\det A \neq 0$, and $\det(A^{-1}) = 1/\det A$.
- $\det(cA) = c^n \det A$ for $n \times n$ (each of n rows scaled).
- Row operations as above; column operations obey the same rules ($\det A^T = \det A$).

Example 7.1 (Via elimination)**.** $A = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 3 & 1 \\ 1 & 0 & 2 \end{bmatrix}$. Swap $R_1 \leftrightarrow R_3$ flips sign: $\det A = -\det \begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 1 \\ 2 & 1 & 0 \end{bmatrix}$. Then $R_3 \leftarrow R_3 - 2R_1$: $-\det \begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 1 \\ 0 & 1 & -4 \end{bmatrix} = -1 \cdot \det \begin{bmatrix} 3 & 1 \\ 1 & -4 \end{bmatrix} = -1 \cdot (-13) = 13.$

7.3 Cofactor Expansion and Adjugate

Delete row i, column j from A to get M_{ij} $(n-1 \times n-1)$. The *cofactor* is $C_{ij} = (-1)^{i+j} \det M_{ij}$. Then for any fixed i,

$$\det A = \sum_{j=1}^{n} a_{ij} C_{ij} \quad \text{(row } i \text{ expansion)},$$

and similarly for any column. The *adjugate* $\operatorname{adj}(A) = [C_{ji}]$ satisfies

$$A \cdot \operatorname{adj}(A) = \det(A)\, I_n,$$

so if $\det A \neq 0$, $A^{-1} = \operatorname{adj}(A)/\det A$. For 2×2, this is $\frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$.

Cofactor expansion is $O(n!)$ — never used for large n; elimination is $O(n^3)$.

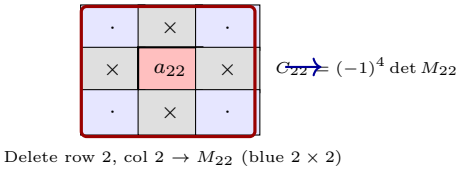

Figure 7.3: Cofactor C_{ij}: delete row i and column j, take the determinant of the remaining $(n-1) \times (n-1)$ minor, multiply by $(-1)^{i+j}$.

7.4 Determinant and Eigenvalues

If $A\mathbf{v} = \lambda\mathbf{v}$, then $\det(A - \lambda I) = 0$ (Ch. 8). Hence $\det A = \prod_i \lambda_i$ (product of eigenvalues) and $\operatorname{tr} A = \sum_i \lambda_i$. Cramer's rule $x_j = \det A_j / \det A$ (replace column j of A by $\mathbf{b}$) follows from the adjugate but is $O(n!)$ — a theoretical tool, not an algorithm.

Example 7.2 (Python — determinants).

```python
import numpy as np
A = np.array([[2.,1.,0.],[0.,3.,1.],[1.,0.,2.]])
print(np.linalg.det(A))          # ~13.0
print(np.linalg.det(A.T))        # same
print(np.linalg.det(A @ A))      # det(A)^2
# Volume scaling: unit cube -> parallelepiped of volume |det A|
print(abs(np.linalg.det(A)))
```

7.5 Multiplicativity and Invertibility Revisited

$\det(AB) = \det A \det B$ implies $\det(A^{-1}) = 1/\det A$ and $\det(PAP^{-1}) = \det A$ (similarity preserves det). Hence det is a basis-free invariant of the linear map $T : \mathbf{x} \mapsto A\mathbf{x}$: it is the volume scaling factor of T, independent of coordinates.

Theorem 7.1 (Volume scaling). For any measurable $S \subset \mathbb{R}^n$, $\operatorname{vol}(A(S)) = |\det A| \cdot \operatorname{vol}(S)$ when A is invertible. $|\det A|$ is the factor by which A stretches n-dimensional volume; $\det A = 0$ collapses volume to zero.

This is why $|\det J|$ appears as the Jacobian in change of variables for integrals.

7.6 Determinant, Rank, and Eigenvalues

$\det A \neq 0 \iff \operatorname{rank} A = n \iff 0$ is not an eigenvalue. More generally, $\det A = \prod_{i=1}^{n} \lambda_i$ where λ_i are eigenvalues (with algebraic multiplicity, possibly complex). So $|\det A|$ is the product of singular values too

(Ch. 8 notes). For triangular A, det is the diagonal product — eigenvalues sit on the diagonal.

Example 7.3 (When det misleads)**.** $A = 0.1\, I_{100}$ has $\det A = 10^{-100}$ (tiny) but A is perfectly invertible ($\kappa = 1$). Determinant magnitude does not measure near-singularity; condition number does. Never test "det ≈ 0" for invertibility in floating point.

7.7 Cramer's Rule and Its Cost

If A invertible, $A^{-1} = \operatorname{adj}(A)/\det A$ and $x_j = \det A_j/\det A$ where A_j replaces column j by $\mathbf{b}$. Cost $O(n!)$ via cofactors vs. $O(n^3)$ via LU: for $n = 20$, $20! \approx 2.4 \times 10^{18}$ — intractable. Cramer is a theoretical gem that proves A^{-1} entries are rational functions of A's entries.

Remark 7.1 (Orientation and handedness). In graphics, $\det > 0$ means a right-handed coordinate frame stays right-handed after A; $\det < 0$ means it flips (mirror). This is why det of a rotation is $+1$ and of a reflection is -1 — the sign is the parity of the transformation.

Example 7.4 (Determinant and invertibility in code)**.** Never branch on `det(A)==0` in floating point; use `np.linalg.matrix_rank` or `cond`. For $n = 100$, round-off makes det under/overflow ($\sim 10^{\pm 100}$) while rank via SVD is reliable. Check `np.linalg.cond(A)` — large means near-singular regardless of det scale.

Example 7.5 (Triangular shortcut)**.** If U is upper-triangular, $\det U = \prod u_{ii}$ immediately. So in $PA = LU$, $\det A = \pm \prod u_{ii}$ (sign = parity of row swaps in P). This is the $O(n^3)$ algorithm; no cofactor needed. For 2×2, both routes agree: $\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$.

7.8 Chapter Notes

Cauchy and Jacobi formalised $\det(AB) = \det A \det B$; Laplace gave the expansion. Modern libraries compute det via LU with partial pivoting and track sign; for large n, $\log|\det|$ is returned to avoid overflow. The volume picture generalises to $|\det|$ as the Jacobian in multivariate calculus.

7.9 Exercises

E7.1 **Compute.** Find $\det A$ for $A = \begin{bmatrix} 3 & 1 & 2 \\ 0 & -1 & 4 \\ 1 & 0 & 1 \end{bmatrix}$ by (a) row operations to triangular and (b) cofactor expansion along row 2. Compare.

E7.2 **Invertibility.** For which c is $A = \begin{bmatrix} 1 & c & 0 \\ c & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix}$ singular? Relate to volume collapse and to eigenvalues.

E7.3 **Rules.** Prove $\det(cA) = c^n \det A$ for $n \times n$ using the multilinear axiom. Use $\det(AB) = \det A \det B$ to show $\det(A^{-1}) = 1/\det A$ when invertible.

E7.4 **Cramer.** Solve $\begin{cases} x + 2y = 5 \\ 3x - y = 1 \end{cases}$ via Cramer's rule and compare with elimination. Why is Cramer $O(n!)$ and not used for $n > 3$?

E7.5 **Coding.** For random $n \times n$ matrices ($n = 10, 50, 100$), compare `np.linalg.det` (via LU) with a cofactor implementation for $n \le 6$ (time and overflow). Plot $\log|\det|$ vs. n.

Chapter 8

Eigenvalues, Eigenvectors, and Diagonalization

"Eigenvectors are the directions a matrix leaves alone — it only stretches them." Finding those directions turns a tangled linear map into independent scalings, powers into powers of scalars, and differential equations into decoupled ones.

From zero: fixed directions before polynomials. We start with "which arrows stay on their own span?" then derive $A\mathbf{v} = \lambda\mathbf{v}$, the characteristic polynomial, and diagonalization. No prior eigen theory assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define eigenvalue/eigenvector and find them via $\det(A - \lambda I) = 0$;

(L2) compute eigenspaces and their dimensions (geometric multiplicity);

(L3) test diagonalizability and construct $A = PDP^{-1}$ when possible;

(L4) use diagonalization to compute A^k, recurrences, and matrix exponentials;

(L5) recognise symmetric matrices' orthogonal diagonalization and the spectral picture.

8.1 Eigenpairs: Directions That Stay Put

A nonzero $\mathbf{v}$ is an *eigenvector* of $A \in \mathbb{R}^{n\times n}$ with *eigenvalue* $\lambda \in \mathbb{R}$ (or $\mathbb{C}$) if

$$A\mathbf{v} = \lambda\mathbf{v}.$$

Geometrically, A stretches $\mathbf{v}$ by λ but does not rotate it off its line. $\mathbf{0}$ is never an eigenvector (but $\lambda = 0$ is allowed — then $A\mathbf{v} = \mathbf{0}$).

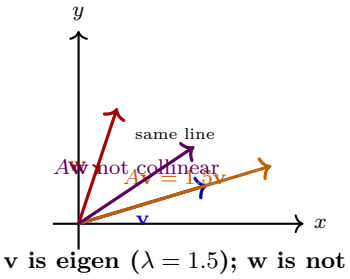

Figure 8.1: An eigenvector stays on its span under A; a generic vector is rotated off. The eigenvalue is the stretch factor (negative flips).

If $A\mathbf{v} = \lambda\mathbf{v}$ then $(A - \lambda I)\mathbf{v} = \mathbf{0}$ with $\mathbf{v} \neq \mathbf{0}$, so $A - \lambda I$ is singular. Hence

Theorem 8.1 (Characteristic equation)**.** λ is an eigenvalue of A iff

$$p_A(\lambda) = \det(A - \lambda I_n) = 0.$$

p_A is the *characteristic polynomial* (degree n, leading term $(-\lambda)^n$ or λ^n up to sign). Its roots are the eigenvalues; $\mathcal{N}(A - \lambda I)$ is the *eigenspace* E_λ.

For 2×2, $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$,

$$p_A(\lambda) = \lambda^2 - \operatorname{tr}(A)\lambda + \det(A), \quad \operatorname{tr} = a + d.$$

So eigenvalues satisfy $\lambda_1 + \lambda_2 = \operatorname{tr} A$, $\lambda_1\lambda_2 = \det A$ — quick checks.

Example 8.1 (A 2×2 shear vs. scaling)**.** $A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$: $p_A(\lambda) = (2 - \lambda)(3 - \lambda)$, so $\lambda = 2, 3$. For $\lambda = 2$, $(A - 2I) = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$, $E_2 = \operatorname{span}\{[1, 0]^T\}$. For $\lambda = 3$, $E_3 = \operatorname{span}\{[1, 1]^T\}$. Two independent eigen-directions — diagonalizable (see §8.3).

8.2 Eigenspaces and Multiplicities

Algebraic multiplicity a_λ: multiplicity of λ as a root of p_A. *Geometric multiplicity* $g_\lambda = \dim \mathcal{N}(A - \lambda I)$: number of independent eigenvectors for λ. Always $1 \leq g_\lambda \leq a_\lambda$. Eigenvectors for distinct λ are independent — so n distinct eigenvalues guarantees n independent eigenvectors.

Example 8.2 (Defective)**.** $J = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$ has $p_J(\lambda) = (2 - \lambda)^2$, so $a_2 = 2$, but $J - 2I = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ has nullity 1, so $g_2 = 1 < 2$. Only one eigen-line — J is not diagonalizable (it is a Jordan block).

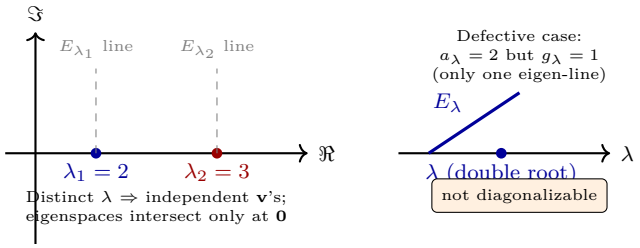

Figure 8.2: Left: distinct eigenvalues give independent eigenspaces. Right: a repeated eigenvalue may have fewer eigenvectors than its algebraic multiplicity — defective, not diagonalizable.

8.3 Diagonalization

A is *diagonalizable* if $A = PDP^{-1}$ with $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ and P invertible. Columns of P are then eigenvectors and diagonal entries are the corresponding eigenvalues.

Theorem 8.2 (Diagonalization criterion)**.** $A \in \mathbb{R}^{n\times n}$ diagonalizable $\iff$ there are n linearly independent eigenvectors $\iff$ for every λ, $g_\lambda = a_\lambda$ (equivalently $\sum_\lambda g_\lambda = n$).

Why care? Powers decouple: $A^k = PD^kP^{-1} = P\operatorname{diag}(\lambda_1^k, \ldots, \lambda_n^k)P^{-1}$. So does $e^{At} = Pe^{Dt}P^{-1}$ and linear recurrences $\mathbf{x}_{k+1} = A\mathbf{x}_k$.

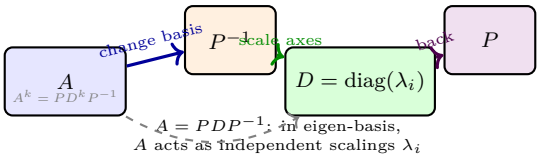

Figure 8.3: Diagonalization as change of basis: P^{-1} moves to eigen-coordinates, D scales each axis by λ_i, P moves back. Powers act entrywise on D.

Example 8.3 (Fibonacci via diagonalization)**.** $\begin{bmatrix} F_{k+1} \\ F_k \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_k \\ F_{k-1} \end{bmatrix}$, so $\mathbf{x}_k = A^k\mathbf{x}_0$. A has eigenvalues $\varphi = (1+\sqrt{5})/2$, $\psi = (1-\sqrt{5})/2$, $A = P\operatorname{diag}(\varphi, \psi)P^{-1}$, giving $F_k = (\varphi^k - \psi^k)/\sqrt{5}$.

Example 8.4 (Python — eigen and diagonalization)**.**

```python
import numpy as np
A = np.array([[2.,1.],[0.,3.]])
w, V = np.linalg.eig(A)          # w eigenvalues, V columns are eigenvectors
print(w)                          # [2., 3.]
print(A @ V[:,0], w[0]*V[:,0])    # check A v = lambda v
# Diagonalization check:
P = V
D = np.diag(w)
print(np.allclose(A, P @ D @ np.linalg.inv(P)))
# Powers:
print(np.allclose(np.linalg.matrix_power(A, 5), P @ np.diag(w**5) @ np.linalg.inv(P)))
```

For symmetric A, `eigh` returns orthonormal Q with $A = Q\Lambda Q^T$ (spectral theorem).

8.4 Symmetric Matrices and the Spectral Theorem

If $A^T = A$ (real symmetric), then all eigenvalues are real, eigenvectors for distinct λ are orthogonal, and A is orthogonally diagonalizable: $A = Q\Lambda Q^T$ with $Q^TQ = I$, Λ real diagonal. This is the *spectral theorem* — the basis that diagonalizes a symmetric matrix is a rotation/reflection. Applications: PCA (covariance is symmetric), graph Laplacians, quadratic forms $Q(\mathbf{x}) = \mathbf{x}^T A\mathbf{x} = \sum \lambda_i y_i^2$.

8.5 Complex Eigenvalues and Real Blocks

Real matrices can have complex eigenpairs when they rotate. $R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ has $\lambda = e^{\pm i\theta}$, eigenvectors $[1, \mp i]^T$ in $\mathbb{C}^2$. Over $\mathbb{R}$, R_θ is not diagonalizable but block-diagonalizable with 2×2 rotation blocks. The real Schur form captures this; the complex Schur form is always triangular.

8.6 Markov Chains and Steady States

A column-stochastic P (nonnegative, columns sum to 1) has $\lambda = 1$ as its largest eigenvalue (Perron–Frobenius); the eigenvector π with $P\pi = \pi$ is the *steady state*. Powers $P^k\mathbf{x}_0 \to \pi$ when P is primitive ($|\lambda_2| < 1$). Example: PageRank is the $\lambda = 1$ eigenvector of the web's link matrix.

Example 8.5 (Two-state chain)**.** $P = \begin{bmatrix} 0.9 & 0.2 \\ 0.1 & 0.8 \end{bmatrix}$: eigenvalues $1, 0.7$, $E_1 = \text{span}\{[2,1]^T\}$, so $\pi = [2/3, 1/3]^T$ after normalising to sum 1. Starting from any distribution, $P^k\mathbf{x}_0 \to \pi$ at rate 0.7^k.

8.7 Singular Value Decomposition (Preview)

Every $A \in \mathbb{R}^{m\times n}$ has $A = U\Sigma V^T$ with U, V orthogonal and Σ diagonal of singular values $\sigma_i = \sqrt{\lambda_i(A^TA)} \geq 0$. Columns of V are eigenvectors of A^TA, of U eigenvectors of AA^T. SVD generalises diagonalization to non-square matrices and powers PCA, compression, and pseudoinverses. If $A = Q\Lambda Q^T$ symmetric, $\sigma_i = |\lambda_i|$.

Remark 8.1 (Diagonalizability vs. invertibility). These are unrelated: $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ is singular and not diagonalizable; $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ is invertible but not diagonalizable; I is both. Test diagonalizability via $g_\lambda = a_\lambda$, not via det.

Example 8.6 (Quadratic forms)**.** For symmetric $A = Q\Lambda Q^T$, $f(\mathbf{x}) = \mathbf{x}^TA\mathbf{x} = \sum \lambda_i y_i^2$ where $\mathbf{y} = Q^T\mathbf{x}$. So $\lambda_i > 0$ for all $i \iff f(\mathbf{x}) > 0$ for $\mathbf{x} \neq \mathbf{0}$ (positive definite) — Ch. 6's A^TA is always positive semidefinite ($\lambda_i \geq 0$).

Example 8.7 (Power iteration intuition)**.** Repeatedly applying A to any $\mathbf{x}_0$ with a component in the dominant eigenspace amplifies the largest $|\lambda|$ direction fastest: $A^k\mathbf{x}_0 \approx c_1\lambda_1^k\mathbf{v}_1$. Normalising each step gives the *power method* for $\lambda_1, \mathbf{v}_1$ — how PageRank and PCA's top component are found when n is huge and full `eig` is too costly.

8.8 Chapter Notes

Eigen (German "own") — Hilbert's term for characteristic values. Diagonalization fails exactly on nontrivial Jordan blocks; over $\mathbb{C}$ every matrix is triangularizable (Schur) and Jordan-form. Numerically, eigenvalues are found via QR iteration (Francis, 1961), not $\det(A - \lambda I)$.

8.9 Exercises

E8.1 **Find eigenpairs.** Compute eigenvalues and eigenspaces for $A = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}$ via $p_A(\lambda) = \lambda^2 - 7\lambda + 10$. Give eigenvectors and verify $A\mathbf{v} = \lambda\mathbf{v}$.

E8.2 **Diagonalize or not?** For $B = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$, find a_λ, g_λ for each λ. Is B diagonalizable? If so give P, D; if not explain why.

E8.3 **Powers.** Let $A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} = PDP^{-1}$ with $D = \operatorname{diag}(3, -1)$. Compute A^4 via PD^4P^{-1} and verify by direct multiplication.

E8.4 **Symmetric.** Show $S = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$ is symmetric, find its eigenvalues $(4, 1, 1)$ and an orthonormal eigenbasis. Write $S = Q\Lambda Q^T$ and $\mathbf{x}^T S\mathbf{x}$ in $y = Q^T\mathbf{x}$ coordinates.

E8.5 **Coding.** For random symmetric 5×5 $A = (M + M^T)/2$, use `np.linalg.eigh` to get Q, Λ and verify $Q^TQ \approx I$, $A \approx Q\Lambda Q^T$, and $\det A \approx \prod \lambda_i$, $\operatorname{tr} A \approx \sum \lambda_i$ numerically.